NeuralOps

AI-Powered Security Operations Center
Detection Engine · Gemini AI Analysis · Real-Time Dashboard

`Python` `FastAPI` `Redis` `Gemini AI` `React` `PostgreSQL`

B.Tech CSE Cybersecurity · VIT Vellore · 2024 Batch · Team Project

---

1. Project Overview

NeuralOps is a full-stack AI-powered Security Operations Center (SOC) that collects logs and network traffic from a target application, detects attacks using a rule-based engine (72 rules, 10 categories), and uses the Gemini API to explain threats in plain English with actionable incident response recommendations — all displayed on a real-time React dashboard.

Core Idea

A rule engine filters 10,000 logs/hour down to ~10 suspicious ones (free, instant). Only those suspicious logs are sent to Gemini AI for deep analysis (fast, cost-efficient). **Rules catch it fast — Gemini explains it smart.**

What NeuralOps Covers

- **SIEM** — Security Information and Event Management — log collection + correlation
- **SOC** — Security Operations Center workflow — alert triage, assign, escalate, close
- **AI** — Gemini API for threat explanation and IR playbook generation
- **Automation** — Real-time detection without manual log reading
- **Web Dev** — Full-stack React + FastAPI dashboard

Key Features

Feature	Description
Real-time alert feed	WebSocket push — alerts appear on dashboard instantly
AI threat explanation	Gemini explains what happened, why it's dangerous, what to do
72 attack patterns	Covering web, auth, network, malware, exfiltration, cloud, AI attacks
MITRE ATT&CK mapping	Each alert tagged with TTP code and kill chain stage
IR playbook	Step-by-step response actions auto-generated by Gemini
Hourly threat report	Gemini summarises all alerts from the past hour
OWASP Top 10	Full detection coverage for all 10 OWASP risk categories

2. System Architecture — 5 Layers

NeuralOps is built as a layered pipeline. Each layer has one job. Data flows top to bottom: raw logs enter at Layer 1 and become an actionable alert on the dashboard at Layer 5.

Layer 1	**Data Ingestion**	*Redis Streams*	Collects logs from FastAPI middleware, nginx access.log, system auth logs (journald), and network packets (scapy). Normalises everything into a JSON envelope and queues it in Redis Streams.
Layer 2	**Detection Engine**	*Python workers*	Three parallel workers: (1) Rule engine — regex patterns for 72 known attack signatures. (2) ML anomaly detector — Isolation Forest flags statistical outliers. (3) Threat intel — AbuseIPDB + VirusTotal API lookup for IPs and file hashes.
Layer 3	**AI Analysis**	*Gemini API*	Only suspicious logs reach this layer. Gemini Flash explains each alert in plain English. Gemini Pro generates an IR playbook using MITRE ATT&CK; context. Hourly scheduled job sends alert summaries for a threat report.
Layer 4	**Backend API**	*FastAPI + PostgreSQL*	REST endpoints for alert CRUD, WebSocket for real-time push to dashboard, PostgreSQL for incident storage, Elasticsearch for full log search. JWT auth with role-based access (analyst / admin / read-only).
Layer 5	**Frontend Dashboard**	*React + Tailwind*	Real-time alert feed with severity colour coding, threat detail view with AI explanation, IR playbook panel, SIEM log explorer, analytics charts (MTTD, MTTR, trends), and incident triage workflow.

```
Data flow: Logs → Redis Streams → Rule Engine → Alert Queue → Gemini API → FastAPI WebSocket → React Dashboard
```

3. Setup & Installation

Prerequisites

- Python 3.10+
- Redis 6+ (running locally or remotely)
- Git

Clone & Setup

```
git clone -b feature/sharan-soc-engine \
    git@github.com:TheWhitehatsclub-vit/AI-Security-OperationsCenter.git
cd AI-Security-Operations-Center
python3 -m venv venv
source venv/bin/activate        # Linux/macOS
pip install -r requirements.txt
```

Install Redis (Ubuntu)

```
sudo apt install python3 python3-pip redis-server git
sudo systemctl start redis
redis-cli ping   # should reply: PONG
```

Configure API Key (Optional — for AI analysis)

Get a free key from **https://aistudio.google.com/apikey** or use Groq (**https://console.groq.com**).

```
export GEMINI_API_KEY="your-key-here"   # Linux/macOS
```

4. Running the System

Option A — Launcher Script (Recommended)

```
./neuralops.sh --all        # Start everything
./neuralops.sh --stop       # Stop all processes
./neuralops.sh --list       # Show running processes
./neuralops.sh --logs       # Tail all logs
./neuralops.sh --engine     # Detection engine only
./neuralops.sh --dashboard  # Dashboard only (port 8002)
./neuralops.sh --gemini     # AI worker only
```

Option B — Manual (Multiple Terminals)

```
# Terminal 1: Redis (already running as service)
redis-server

# Terminal 2: Detection Engine
python3 workers/detect_engine.py

# Terminal 3: Dashboard
python3 gemini/dashboard.py

# Terminal 4: AI Worker (optional)
python3 gemini/worker.py
```

Component Summary

Script	What it does
`collectors/middleware.py`	Captures HTTP requests as FastAPI middleware → neuralops:logs
`collectors/scapy.py`	Monitors /proc/net/tcp for port scans, ARP spoofing (Linux only)
`collectors/journald.py`	Reads systemd journal for SSH/sudo/auth failures (Linux only)
`workers/detect_engine.py`	Main engine: reads all 3 streams, matches 72 rules, emits alerts
`workers/ml_anomaly.py`	Isolation Forest anomaly detection, retrains every 120s
`workers/threat_intel.py`	Enriches alerts with AbuseIPDB, GreyNoise, VirusTotal
`workers/output.py`	Writes alerts to log file + optional Slack webhook
`gemini/worker.py`	AI analysis: summary, severity reassessment, remediation steps
`gemini/dashboard.py`	SOC web dashboard on port 8002 — live alerts + AI analysis

5. Log Collection Methods

Method 1 — FastAPI Middleware — Easiest

Add a middleware function to your FastAPI app. Every HTTP request is automatically logged and pushed to Redis. 20 lines of code, works immediately.

```
pip install redis

@app.middleware('http')
async def log_requests(request, call_next):
    body = await request.body()
    response = await call_next(request)
    r.xadd('neuralops:logs', {
        'ip': request.client.host,
        'path': request.url.path,
        'body': body.decode()[:500],
        'status': response.status_code
    })
```

Method 2 — Tail Log Files — Easy

Python reads /var/log/nginx/access.log and /var/log/auth.log in real time using a tail loop. No agents needed.

```
with open('/var/log/nginx/access.log') as f:
    f.seek(0, 2)          # go to end
    while True:
        line = f.readline()
        if line:
            r.xadd('neuralops:logs', {'raw': line})
        else:
            time.sleep(0.1)
```

Method 3 — Network Packet Capture — Medium

scapy captures raw packets from the network interface. Detects port scans, SYN floods, ARP poisoning — attacks that never appear in application logs.

```
from scapy.all import sniff, IP, TCP

def process(pkt):
    if IP in pkt and TCP in pkt:
        r.xadd('neuralops:network', {
            'src': pkt[IP].src,
            'dst': pkt[IP].dst,
            'flags': str(pkt[TCP].flags)
        })

sniff(iface='eth0', prn=process, store=False)
```

Method 4 — System Logs via journald — Medium

Reads SSH, sudo, and auth events from Linux system journal. Catches brute force SSH, privilege escalation, new user creation.

```
from systemd import journal
j = journal.Reader()
j.seek_tail()
j.add_match(_SYSTEMD_UNIT='ssh.service')
for entry in j:
    r.xadd('neuralops:system', {'msg': entry['MESSAGE']})
```

6. Attack Pattern Library — 72 Rules

NeuralOps detects attacks across 10 categories. Each pattern is a Python rule — a regex match, a threshold check, or an anomaly condition.

Category	Example Patterns	Count
SQL Injection	OR bypass, UNION SELECT, stacked queries, blind SQLi	10
XSS	Reflected, stored, DOM-based, SVG payloads	5
Command Injection	Shell metacharacters, backtick exec, pipe chains	8
SSRF	Internal metadata, cloud IMDS, localhost bypass	4
Path Traversal	../ sequences, null bytes, URL encoding	6
File Inclusion	LFI/RFI patterns	2
LDAP / XXE	LDAP injection, external entity references	4
API Key Leak	Keys in query params, Authorization headers, logs	3
Network-level	Port scan, ARP spoofing, SYN flood, scanner fingerprints	18
System / Auth	SSH brute force, sudo abuse, auth failures, priv escalation	8
Web (misc)	Webshell upload, Log4Shell, C2 beaconing, prompt injection	4

Rule Structure (Python)

```
RULES = {
    'SQLi': {
        'pattern':  r"(?i)(' OR|UNION SELECT|--|;DROP)",
        'fields':   ['url', 'body'],
        'severity': 'CRITICAL',
        'ttp':      'T1190',   # MITRE ATT&CK
    },
    'BruteForce': {
        'threshold': {'status': 401, 'count': 10, 'window': 60},
        'group_by':  'source_ip',
        'severity':  'HIGH',
        'ttp':       'T1110',
    },
}
# When a rule fires → alert sent to Gemini for explanation
```

7. Gemini AI Integration

Gemini is the brain of NeuralOps. It never sees raw logs directly — only the alerts that the rule engine has already flagged. This keeps API calls minimal and responses fast and focused.

Threat Explainer (gemini-1.5-flash)

Called once per alert. Takes the flagged log + matched rule as input. Returns a plain-English explanation: what happened, why it's dangerous, what the attacker likely wanted. Fast and cheap.

IR Playbook Generator (gemini-1.5-pro)

Called for HIGH and CRITICAL alerts. Uses MITRE ATT&CK; context to generate a step-by-step incident response checklist specific to the attack type. More detailed reasoning required.

Hourly Threat Report (Scheduled job)

Every hour, collect the last hour's alert summaries and send to Gemini. Returns a paragraph-level threat briefing for the SOC team. The only time Gemini sees multiple alerts at once.

Why not send all logs to Gemini?

10,000 logs/hour → 10,000 API calls → rate limit hit in minutes, free tier exhausted instantly, 5–10 min dashboard lag. Rule engine filters first → only ~10 suspicious logs/hour reach Gemini → real-time, free-tier friendly, pinpoint accurate.

Gemini API Call (Python)

```
import google.generativeai as genai
import asyncio, json

genai.configure(api_key=os.environ['GEMINI_API_KEY'])
model = genai.GenerativeModel('gemini-1.5-flash')

async def explain_threat(alert: dict) -> str:
    prompt = f"""
    You are a SOC analyst. Explain this alert in plain English.
    What happened, why is it dangerous, what should the analyst do?
    Alert: {json.dumps(alert)}
    Keep it under 150 words. Be direct."""
    response = await asyncio.to_thread(model.generate_content, prompt)
    return response.text
```

8. Detection Engine — Core Pipeline

The detection engine is the heart of NeuralOps. It normalizes, enriches, matches, scores, and deduplicates every incoming log before emitting a structured alert to Redis.

Pipeline Steps

- `normalize_log()` — Flattens JSON, URL-decodes query strings, extracts base64 blobs, HTML-decodes entities
- `enricher.enrich()` — GeoIP lookup (MaxMind GeoLite2) → country, city, ASN, ISP, is_private flag
- `baseline.observe()` — Per-IP behavioral profiler: tracks methods, paths, UAs, statuses, body sizes, timing
- `match_rule()` — 72 rules scanned — regex, threshold, time-based, correlation, geo-regex matchers
- `compute_confidence()` — Fuses rule score + baseline anomaly + chain boost + obfuscation detection
- `dedup.check()` — Fingerprint-based dedup with 300s TTL window to suppress repeated identical alerts
- `correlator.check()` — 5 attack chain patterns (recon→exploit→install) — boosts confidence on sequences
- `emit alert` — Structured JSON pushed to neuralops:alerts Redis stream for dashboard + Gemini consumption

Sample Alert JSON Output


```
{
  "id":               "1607a4c2-07e3-488a-b2fc-aba8256ee198",
  "timestamp":        "2026-06-23T13:54:06Z",
  "rule_name":        "XSS",
  "category":         "web",
  "severity":         "MEDIUM",
  "confidence":       0.402,
  "confidence_boost": 0.05,
  "confidence_factors": [["obfuscation", 0.05]],
  "ttp":              "T1190",
  "kill_chain":       "Exploitation",
  "source_ip":        "127.0.0.1",
  "matched_value":    "onmouseover=\"alert(1)\"",
  "matched_pattern":  "(?i)<script[^>]*>.*?</script>",
  "description":      "XSS detected",
  "geo": {"country": "", "city": "", "is_private": true},
  "raw_log": { ... }          // original log fields preserved
}
```


Redis Stream Architecture

Stream	Direction	Purpose
`neuralops:logs`	← Collectors write	Raw incoming HTTP / network / system logs
`neuralops:alerts`	→ Engine emits	Enriched alert JSONs (UI team reads this)
`neuralops:incidents`	→ Engine emits	Grouped alerts (10+ same rule+IP in 60s)
`neuralops:fp_feedback`	← UI writes	False positive flags from analysts
`neuralops:rules_config`	→ Engine reads	Hot-reload rules (checked every 30s)
`neuralops:state:*`	↔ Engine R/W	Baseline + threshold state, saved every 60s

9. Full Technology Stack

Component	Technology	Why
Language	Python 3.11+	Detection engine, API, collectors
Web framework	FastAPI	Async, WebSocket support, familiar
Message broker	Redis Streams	Lightweight Kafka alternative
Primary database	PostgreSQL	Structured incident and alert data
Log search	Elasticsearch	Full-text search over raw logs
ML detection	scikit-learn	Isolation Forest for anomaly detection
Network capture	scapy / pyshark	Packet-level attack detection
AI analysis	Gemini API	Threat explanation + IR playbooks
Vector store	ChromaDB / FAISS	MITRE ATT&CK RAG lookups
Frontend	React + Tailwind CSS	Dashboard, real-time UI
Charts	Recharts / D3.js	Alert trends, heatmaps
Real-time push	WebSocket (FastAPI)	Live alert feed to dashboard
Auth	JWT + RBAC	Analyst / admin / read-only roles
Containerisation	Docker + docker-compose	One-command project setup
Demo target	DVWA / Juice Shop	Vulnerable app to fire attacks at

10. Dashboard Views & API Endpoints

Dashboard Views

View	What it shows
Command Center	Alert count, active rules, AI triage count, activity feed, category pie chart, timeline
Alert Feed	Filterable alert list (by severity/source), detail panel with full alert context + AI analysis
Intelligence	Gemini/Groq AI feed — summaries, severity, FP likelihood, recommended actions
Systems	Detection rules count, collector status, input streams, uptime

API Endpoints

Endpoint	Method	Description
`/api/stats`	GET	Alert statistics, counts, analysis totals
`/api/alerts?limit=N`	GET	Recent alerts (paginated)
`/api/analysis?limit=N`	GET	AI analysis results
`/api/stream`	GET	SSE live stream for real-time updates
`/api/feedback`	POST	Submit false positive feedback
`/health`	GET	Engine health check

11. Troubleshooting

Problem	Fix
Redis connection refused	sudo systemctl start redis or redis-server
Dashboard blank	Check port 8002 is free: fuser -k 8002/tcp then restart
Gemini API errors	Key quota exhausted — wait or switch to Groq
No alerts appearing	Run scripts/simulate_attacks.py to generate test data
Module not found	Activate venv: source venv/bin/activate
Permission denied (Linux)	chmod +x neuralops.sh

Docker Deployment

```
docker-compose up -d   # starts Redis + Engine + Dashboard + AI Worker in containers
```

Redis Quick Reference

```
# Count alerts received
redis-cli XLEN neuralops:alerts

# Read last 5 alerts (full JSON)
redis-cli XREAD COUNT 5 STREAMS neuralops:alerts 0

# Watch alerts in real-time
redis-cli XREAD COUNT 1 BLOCK 0 STREAMS neuralops:alerts $

# Clear data for fresh demo
redis-cli DEL neuralops:logs neuralops:alerts neuralops:incidents
```